

Практическое задание 2

Методы программирования в реальном времени для Linux

<https://github.com/ananass12/Real-time-systems>

Основы времени и периодических задач

Задание 1

Задание 1: Анализ системных часов (`calctime1.c`, `calctime2.c`)

- **Цель:** Понять разницу между `CLOCK_REALTIME` и `CLOCK_MONOTONIC`, научиться создавать стабильный периодический цикл с помощью `clock_nanosleep`.
- **Действия:**
 1. Изучите `calctime1.c` (предоставлен для анализа). Обратите внимание, как измеряется разрешение часов.
 2. Модифицируйте `calctime2.c`:
 - Реализуйте цикл, который просыпается каждые 2 миллисекунды с использованием `clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, ...)`.
 - Собирайте статистику реальных интервалов между пробуждениями (дельта).
 - Рассчитайте и выведите не только min/max/avg, но и **стандартное отклонение**, чтобы оценить стабильность периода.
 3. В комментариях к коду объясните, почему `TIMER_ABSTIME` критически важен для предотвращения накопления ошибки (дрейфа таймера).

АНАЛИЗ CALCTIME1.C

Программа не использует системный вызов `clock_getres()` (который напрямую возвращает теоретическое разрешение часов), а вместо этого эмпирически оценивает его:

- Считывается текущее значение времени
- В цикле повторно вызывается `clock_gettime`, пока возвращаемое значение не изменится

```
clock_gettime(CLOCK_REALTIME, &prevclockval);
do {
    clock_gettime(CLOCK_REALTIME, &clockval);
} while (clockval.tv_sec == prevclockval.tv_sec &&
        clockval.tv_nsec == prevclockval.tv_nsec);
```

- Программа ждёт первого изменения системных часов
- Вычисляется разница между двумя последовательными

$\text{delta} = (\text{новое время в нс}) - (\text{предыдущее время в нс});$

- Этот процесс повторяется NumSamples раз (10 раз), чтобы получить несколько измерений минимального шага.

CLOCK_REALTIME — часы реального времени, действующие в пределах всей системы. Установка этих часов требует специальных привилегий.

CLOCK_MONOTONIC — часы с монотонным отсчетом времени, которые не может устанавливать какой-либо процесс. Представляют время, истекшее с какого-либо не жестко задаваемого момента в прошлом (например, с момента загрузки системы).

МОДИФИКАЦИЯ CALCTIME1.C

```
int64_t next_ns = timespec_to_ns(&t_next) + period_ns; //стартуем через один период
for (samples = 0; samples < NUM_SAMPLES; ++samples) {
    ns_to_timespec(next_ns, &t_next);

    //Абсолютный сон до t_next
    /* TIMER_ABSTIME - спать ДО этого момента, а не спать ЕЩЁ N нс.
    Это предотвращает накопление ошибки - даже если один раз проспали,
    следующее пробуждение всё равно будет по идеальному графику
    */
    int rc;
    do {
        rc = clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &t_next, NULL);
    } while (rc == EINTR);
    if (rc != 0) {
        fprintf(stderr, "clock_nanosleep failed: %s\n", strerror(rc));
        return EXIT_FAILURE;
    }

    if (clock_gettime(CLOCK_MONOTONIC, &now) != 0) {
        fprintf(stderr, "clock_gettime failed: %s\n", strerror(errno));
        return EXIT_FAILURE;
    }

    int64_t now_ns = timespec_to_ns(&now);
    //Интервал = текущее время - время предыдущего пробуждения
    //Предыдущее пробуждение было запланировано на (next_ns - period_ns)
    deltas_ns[samples] = now_ns - (next_ns - period_ns);

    next_ns += period_ns;
}
```

Сбор статистики:

```
//Статистика
int64_t min_ns = INT64_MAX, max_ns = INT64_MIN, sum_ns = 0;
for (int i = 0; i < NUM_SAMPLES; ++i) {
    if (deltas_ns[i] < min_ns) min_ns = deltas_ns[i];
    if (deltas_ns[i] > max_ns) max_ns = deltas_ns[i];
    sum_ns += deltas_ns[i];
}
double avg_ns = (double)sum_ns / (double)NUM_SAMPLES;
```

Стандартное отклонение:

```
//Расчёт стандартного отклонения — показывает стабильность периода
double sum_sq_diff = 0.0;
for (int i = 0; i < NUM_SAMPLES; ++i) {
    double diff = (double)deltas_ns[i] - avg_ns;
    sum_sq_diff += diff * diff;
}
double std_dev_ns = sqrt(sum_sq_diff / NUM_SAMPLES);
```

Задание 2

Задание 2: Периодический таймер на `timerfd` (`reptimer_timerfd.c`)

- Цель: Освоить современный Linux API (`timerfd`) для создания периодических событий без использования сигналов.
- Действия:
 1. Реализуйте программу, которая создает таймер `timerfd`.
 2. Настройте его на первое срабатывание через 5 секунд, а затем периодически каждые 1500 мс.
 3. В цикле ожидайте событий от таймера с помощью `read()` и выводите количество истечений и текущее время.
 4. Сравните в комментариях к коду подходы с `clock_nanosleep` и `timerfd`. В каких случаях `timerfd` предпочтительнее? (Подсказка: интеграция с `epoll`).

`timerfd` — это специальный файловый дескриптор в Linux, который позволяет приложениям отслеживать время, а также получать уведомления о его истечении с помощью событий ввода-вывода. Он эмулирует поведение таймера, позволяя приложению считывать количество сработавших таймеров с момента последнего чтения.

НАСТРОЙКА СРАБАТЫВАНИЙ:

```
//Настройка времени: первое срабатывание
its.it_value.tv_sec = 5;
its.it_value.tv_nsec = 0;

//Настройка времени: период после первого срабатывания
its.it_interval.tv_sec = 1;
its.it_interval.tv_nsec = 500000000; // 1.5 секунды
```

```

printf("Timer configured. Waiting for %d expirations...\n", iterations_to_run);
for (int i = 0; i < iterations_to_run; ++i) {
    //read() блокируется, пока таймер не сработает
    //Возвращает 8 байт (uint64_t), содержащих количество истечений таймера
    //Если система не сильно нагружена, это значение будет 1
    ssize_t rd = read(tfd, &expirations, sizeof(expirations));
    if (rd < 0) {
        if (errno == EINTR) { // Прервано сигналом, можно проигнорировать
            --i;
            continue;
        }
        perror("read(timerfd) failed");
        close(tfd);
        return EXIT_FAILURE;
    }

    if (clock_gettime(CLOCK_MONOTONIC, &now) != 0) {
        perror("clock_gettime failed");
        close(tfd);
        return EXIT_FAILURE;
    }
    printf("Timer expired! Timestamp: [%ld.%09ld], expirations counter: %" PRIu64 "\n",
        now.tv_sec, now.tv_nsec, expirations);
}

```

Функция `clock_nanosleep()` работает аналогично `nanosleep()`. Функция `nanosleep()` переводит вызывающий процесс в спящий режим на период, указанный в `req`, а потом возвращает 0. При ошибке функция возвращает `-1` и присваивает `errno` соответствующее значение.

СРАВНЕНИЕ ПОДХОДОВ:

1. `clock_nanosleep`:

- поток полностью останавливается
- минимальный код для ожидания
- гарантированное время сна
- только однократное срабатывание

2. `timerfd` + `epoll`:

- основной поток продолжает работу
- интеграция с `epoll` - можно обрабатывать `multiple events`
- один поток обрабатывает множество таймеров и других событий
- поддержка повторяющихся событий

`timerfd` предпочтительнее для асинхронных приложений с `event loop` и приложений с множественными таймерами

Таймауты, планировщик и техники минимизации задержек

Задание 3

Задание 3: Стратегии обработки таймаутов (`timeout_*.c`)

- Цель: Изучить три разных способа организации ожидания с таймаутом.
- Действия:
 1. Изучите и соберите `timeout_poll.c` (ожидание на файловом дескрипторе), `timeout_condvar.c` (межпоточное ожидание) и `timeout_mq.c` (межпроцессное ожидание).
 2. Реализуйте `timeout_ppoll.c`. В нем создайте обработчик сигнала (например, `SIGUSR1`), заблокируйте этот сигнал с помощью `sigprocmask`. Затем используйте `ppoll`, чтобы атомарно разблокировать сигнал на время ожидания события от файлового дескриптора. Это продемонстрирует безопасную обработку сигналов.
 3. Опишите в комментариях к коду ключевые различия в API и сценарии применения каждого из четырех механизмов.

В чем основная проблема при комбинации сигналов и функций ожидания (например, `read`, `poll`)?

- Race Condition: Если сигнал придет после проверки условия, но до вызова `read()/poll()`, поток уйдет в блокировку навсегда, так как сигнал уже обработан и больше не прервет системный вызов.

Как `ppoll` решает проблему race condition с сигналами?

Атомарность: `ppoll` принимает маску сигналов (`sigmask`). Внутри одного системного вызова он:

- Временно заменяет маску сигналов текущего потока на переданную.
- Проверяет готовность дескрипторов.
- Если готовности нет, уходит в ожидание с уже установленной новой маской.
- После выхода из ожидания восстанавливает исходную маску.
- Это гарантирует, что сигналы, разблокированные в `sigmask`, либо придут до начала ожидания (и обработаются), либо прервут ожидание. Потери сигнала не произойдет.

РЕАЛИЗАЦИЯ TIMEOUT_PPOLL.C

`sigprocmask()` — это системная функция, которая используется для блокировки и/или разблокировки сигналов для текущего процесса или потока. Она позволяет проверить текущую маску сигналов (набор заблокированных сигналов) и изменить ее, указав, какие сигналы следует заблокировать, разблокировать или установить как текущую маску.

```
//2.Блокируем SIGUSR1 с помощью sigprocmask
sigset_t blocked_mask, original_mask;
sigemptyset(&blocked_mask);
sigaddset(&blocked_mask, SIGUSR1);
if (sigprocmask(SIG_BLOCK, &blocked_mask, &original_mask) != 0) {
    perror("pthread_sigmask");
    return EXIT_FAILURE;
}
printf("Main thread blocked SIGUSR1.\n");
```

Основные отличия API:

1. `ppoll()` и `poll()`:
 - `ppoll()` принимает дополнительный параметр `const sigset_t *sigmask`.
 - Атомарно заменяет маску сигналов на время ожидания
 - Исключает race condition между разблокировкой сигналов и началом ожидания
2. `sigprocmask()` и `pthread_sigmask()`:
 - `sigprocmask()` работает на уровне процесса (все потоки)
 - `pthread_sigmask()` работает на уровне отдельных потоков
 - В однопоточном приложении разницы нет

СРАВНЕНИЕ ЧЕТЫРЕХ МЕХАНИЗМОВ ОБРАБОТКИ СИГНАЛОВ И ОЖИДАНИЯ:

1. `signal()` + `read()/poll()`:

- Возможна race condition - сигнал может прийти до вызова `read()/poll()` и будет потерян, что приведет к вечному блокированию
- API: `signal(signum, handler); read(fd, buf, size);`
- Использование - простые приложения, где потеря сигнала не критична.

2. `sigaction()` + `read()/poll()` с `SA_RESTART`:

- Сигнал прерывает системный вызов, но `SA_RESTART` автоматически перезапускает его. Однако не все системные вызовы поддерживают перезапуск.
- API: `sigaction()` с флагом `SA_RESTART`; `read()/poll()`;
- Использование - когда нужно автоматическое восстановление после сигнала.

3. `sigprocmask()` + `sigwait()` + `poll()`:

- Сигналы блокируются и обрабатываются синхронно через `sigwait()` в отдельном цикле или потоке.
- API: `sigprocmask()` -> `poll()` -> `sigwait()` в цикле

- Использование - серверы, которые должны обрабатывать сигналы и события от сокетов детерминированным образом.

4. sigprocmask() + ppoll()/pselect():

- Атомарное разблокирование сигналов на время ожидания, что исключает race condition.

Сигнал либо приходит ДО начала ожидания (и обрабатывается), либо ВО ВРЕМЯ ожидания (прерывает его).

- API: sigprocmask() -> ppoll() с передачей маски разблокированных сигналов

- Использование - критические приложения, где нельзя потерять ни один сигнал и нужна максимальная надежность.

Задание 4

Задание 4: Оптимизация для реального времени (sched_fifo_jitter.c)

- Цель: Измерить джиттер планировщика и применить техники для его уменьшения.
- Действия:
 1. Сначала запустите sched_fifo_jitter.c без изменений (под обычным планировщиком SCHED_OTHER). Замерьте и запишите статистику джиттера.
 2. Модифицируйте код, добавив в начало main():
 - Переключение планировщика: sched_setscheduler для установки политики SCHED_FIFO с высоким приоритетом (например, 50).
 - Блокировка памяти: mlockall(MCL_CURRENT | MCL_FUTURE) для предотвращения откачки памяти процесса в swap.
 - Привязка к ядру CPU: pthread_setaffinity_np для закрепления потока на одном ядре (например, последнем доступном ядре в системе), чтобы избежать миграции.
 3. Запустите модифицированную версию (скорее всего, потребуются права root или специальные capabilities: sudo ./bin/sched_fifo_jitter).
 4. Сравните в комментариях к коду результаты "до" и "после". Объясните, как каждая из трех техник (планировщик, блокировка памяти, привязка к ядру) способствует уменьшению джиттера.

Джиттер - отклонение фактического времени реакции или выполнения от ожидаемого.

Вред - система реального времени должна быть предсказуемой.

Большой джиттер означает, что время отклика недетерминировано, что может привести к сбоям в критических процессах.

Как три техники из Задания 4 помогают уменьшить джиттер?

- SCHED_FIFO (Планировщик реального времени): Задача с высоким приоритетом не вытесняется задачами SCHED_OTHER. Это гарантирует, что как только задача проснется или станет готова к выполнению, она будет запущена немедленно.
- mlockall() (Блокировка памяти): Предотвращает вытеснение страниц памяти на диск (swap) и page faults при обращении к новым страницам. Page faults могут приводить к задержкам в миллисекунды, что недопустимо в РВС.

- `pthread_setaffinity_np()` (Привязка к ядру CPU): Предотвращает миграцию потока между ядрами. Это сохраняет кеш процессора (L1/L2) и TLB (буфер ассоциативной трансляции), доступ к которым при миграции теряется, что вызывает задержки.

ЗАПУСК `SCHED_FIFO_JITTER.C` БЕЗ ИЗМЕНЕНИЙ:

```
● nastya@Ubuntu:~/RTS/tasks/task2$ ./bin/sched_fifo_jitter
Switched to SCHED_FIFO priority 50
Pinned thread to CPU 11

Jitter statistics over 5000 samples (2ms period):
  min latency: 7643 ns
  avg latency: 83420.6 ns
  99th percentile: 416081 ns
  max latency: 631228 ns
○ nastya@Ubuntu:~/RTS/tasks/task2$
```

ИЗМЕНЕНИЕ КОДА:

```
int main(void) {

    //1. Установка политики планировщика SCHED_FIFO
    // Переводим поток в режим реального времени с приоритетом 50.
    // Это позволяет потоку вытеснять все обычные (SCHED_OTHER) задачи.
    struct sched_param sp = {.sched_priority = 50};
    if (sched_setscheduler(0, SCHED_FIFO, &sp) != 0) {
        perror("ПРЕДУПРЕЖДЕНИЕ: не удалось установить SCHED_FIFO; продолжаем с обычным планировщиком");
    } else {
        printf("Установлена политика SCHED_FIFO с приоритетом %d\n", sp.sched_priority);
    }

    //2. Блокировка всей памяти процесса
    // mlockall(MCL_CURRENT | MCL_FUTURE) предотвращает выгрузку страниц памяти в swap.
    // Page fault во время критического участка может вызвать задержки в миллисекунды.
    if (mlockall(MCL_CURRENT | MCL_FUTURE) != 0) {
        perror("ПРЕДУПРЕЖДЕНИЕ: не удалось заблокировать память (mlockall)");
    } else {
        printf("Память процесса заблокирована (нельзя выгружать в swap)\n");
    }

    //3. Привязка потока к одному ядру CPU
    // Предотвращает миграцию потока между ядрами, что сохраняет кэш и TLB, уменьшая латентность и джиттер.
    long n_cpus = sysconf(_SC_NPROCESSORS_ONLN);
    if (n_cpus > 0) {
        cpu_set_t cpu_set;
        CPU_ZERO(&cpu_set);
        // Привязываем к последнему ядру – оно часто менее загружено системными задачами.
        CPU_SET(n_cpus - 1, &cpu_set);
        if (pthread_setaffinity_np(pthread_self(), sizeof(cpu_set), &cpu_set) != 0) {
            perror("ПРЕДУПРЕЖДЕНИЕ: не удалось установить привязку к CPU");
        } else {
            printf("Поток привязан к ядру CPU %ld\n", n_cpus - 1);
        }
    }
}
```

ЗАПУСК `SCHED_FIFO_JITTER.C` ПОСЛЕ ИЗМЕНЕНИЙ:


```

● nasty@Ubuntu:~/RTS/tasks/task2$ ./bin/sched_fifo_jitter
Установлена политика SCHED_FIFO с приоритетом 50
Память процесса заблокирована (нельзя выгружать в swap)
Поток привязан к ядру CPU 11

Статистика джиттера по 5000 измерениям (период 2 мс):
Минимальная задержка: 12077 нс
Средняя задержка: 60773.7 нс
99-й перцентиль: 352111 нс
Максимальная задержка: 607128 нс
● nasty@Ubuntu:~/RTS/tasks/task2$

```

СРАВНЕНИЕ ДО/ПОСЛЕ:

До:

Джиттер мог достигать десятков или сотен микросекунд из-за:

- вытеснения обычным планировщиком,
- page faults при обращении к новой памяти,
- миграции между ядрами с потерей кэша.

После:

- SCHED_FIFO гарантирует немедленное пробуждение (если нет других RT-потоков)
- mlockall устраняет page faults - нет задержек от диска/swap.
- Привязка к CPU сохраняет кэш L1/L2 и TLB - стабильное время доступа к памяти
- Джиттер обычно снижается до сотен или даже десятков наносекунд

Контрольные вопросы

1. **Сравнение clock_nanosleep и nanosleep:** Почему для периодических задач в CPB clock_nanosleep с абсолютным временем всегда предпочтительнее, чем относительный nanosleep? Проиллюстрируйте проблему накопления ошибки на гипотетическом примере.
2. **timerfd vs Сигналы:** POSIX-таймеры (timer_create) могут доставлять события через сигналы. Почему timerfd считается более надежным и предсказуемым механизмом для построения циклов обработки событий (event loops)?
3. **Инверсия приоритетов:** Хотя мы не реализовывали это напрямую, опишите сценарий, в котором использование pthread_mutex может привести к инверсии приоритетов. Какие атрибуты мьютекса (см. pthread_mutexattr_setprotocol) помогают решить эту проблему?
4. **Предсказуемость vs Производительность:** Объясните, почему техники, использованные в задании 4 (SCHED_FIFO, привязка к ядру), могут уменьшить общую производительность системы, но при этом повысить ее предсказуемость.
5. **Жесткое реальное время:** Достаточно ли рассмотренных техник для построения системы жесткого реального времени на стандартном ядре Linux? Что такое PREEMPT_RT и какие фундаментальные изменения в ядре он вносит для обеспечения детерминизма?

1. Сравнение clock_nanosleep и nanosleep

Для периодических задач в системах реального времени clock_nanosleep с абсолютным временем всегда предпочтительнее относительного nanosleep из-за проблемы накопления ошибки.

При использовании nanosleep с относительным временем каждая итерация цикла включает время выполнения полезной работы плюс время ожидания. Например, если задача должна выполняться каждые 100ms, а полезная работа занимает 5ms, то с nanosleep последовательность будет: работа(5ms) + ожидание(100ms) = период 105ms. На следующей итерации: работа(5ms) + ожидание(100ms) = общее

время 210ms от начала, хотя должно быть 200ms. Эта ошибка в 5ms накапливается с каждой итерацией, приводя к дрейфу временной шкалы.

Clock_nanosleep с абсолютным временем решает эту проблему, вычисляя следующее время активации как абсолютное значение. После выполнения работы задача просто ожидает до заранее вычисленного абсолютного времени следующей активации, что гарантирует точный период независимо от времени выполнения работы.

2. timerfd vs Сигналы

timerfd считается более надежным механизмом для построения циклов обработки событий по нескольким причинам:

- Единая модель программирования - timerfd интегрируется с стандартными механизмами ввода-вывода (select, poll, epoll), позволяя обрабатывать таймеры как обычные файловые дескрипторы вместе с сокетами и каналами.
- Отсутствие асинхронных прерываний – в отличие от сигналов, которые прерывают выполнение программы, timerfd генерирует события в основном цикле событий, что упрощает обработку ошибок и отладку.
- Избегание проблем с reentrancy - обработчики сигналов имеют серьезные ограничения на используемые функции, в то время как обработка timerfd в основном цикле не имеет таких ограничений.
- Более предсказуемая доставка -при высокой нагрузке сигналы могут теряться или объединяться, в то время как timerfd гарантирует точный учет срабатываний через счетчик в файловом дескрипторе.

3. Инверсия приоритетов

Сценарий инверсии приоритетов возникает, когда низкоприоритетная задача захватывает мьютекс, необходимый высокоприоритетной задаче, но сама вытесняется задачей со средним приоритетом. Например: задача L (низкий приоритет) захватывает мьютекс M. Задача H (высокий приоритет) пытается захватить M и блокируется. Задача M (средний приоритет) начинает выполняться и не дает задаче L завершить работу с мьютексом M, тем самым косвенно блокируя задачу H.

Для решения этой проблемы в pthreads существуют следующие атрибуты мьютекса через pthread_mutexattr_setprotocol:

- PTHREAD_PRIO_NONE - стандартное поведение без управления приоритетами
- PTHREAD_PRIO_INHERIT - при блокировке высокоприоритетной задачи владелец мьютекса временно повышает свой приоритет до приоритета самой высокой заблокированной задачи
- PTHREAD_PRIO_PROTECT - владелец мьютекса всегда выполняется с заданным приоритетом защиты, независимо от других задач

4. Предсказуемость vs Производительность

Техники SCHED_FIFO и привязки к ядрам повышают предсказуемость за счет общей производительности. Причины:

- Отсутствие вытеснения по времени - в SCHED_FIFO задача выполняется до явной блокировки или завершения, что может привести к голоданию других задач и снижению общей пропускной способности системы.
- Привязка к ядрам - исключает преимущества миграции задач между ядрами для балансировки нагрузки, что может привести к неоптимальному использованию ресурсов многоядерной системы.
- Блокировка кэша - привязка к ядру улучшает локальность кэша для конкретной задачи, но предотвращает динамическую оптимизацию распределения нагрузки.
- Нарушение fair sharing - политики реального времени игнорируют принципы справедливого распределения процессорного времени, что ухудшает отзывчивость фоновых и интерактивных задач.

5. Жесткое реальное время и PREEMPT_RT

Основные ограничения стандартного ядра:

- Невытесняемые секции ядра - длинные критические секции в ядре, где вытеснение отключено, могут вызывать задержки в сотни микросекунд.
- Spinlocks в пространстве ядра - блокировки вращения могут удерживаться длительное время.
- Прерывания - обработчики прерываний имеют высший приоритет и могут задерживать выполнение задач реального времени.

PREEMPT_RT (Real-Time) вносит изменения:

- Полная вытесняемость ядра - большинство критических секций становятся вытесняемыми.
- Замена spinlocks на мьютексы - блокировки вращения преобразуются в вытесняемые мьютексы.
- Threaded interrupts - обработчики прерываний выполняются как обычные потоки ядра с управляемыми приоритетами.
- Priority inheritance - реализация наследования приоритетов для всех примитивов синхронизации.
- Детерминированные системные вызовы - гарантированное время выполнения системных вызовов.